

Quiz 4 Solution

Q1. What is function overriding in C++?

- A) Declaring multiple functions with the same name but different parameters
- B) Redefining a function in a derived class that has the same signature as a `virtual` function in the base class
- C) Using a function inside another function
- D) Defining a function in a class without implementing it

Correct Answer: B

Q2. What is the purpose of the `virtual` keyword in function overriding?

- A) It prevents function overriding
- B) It allows dynamic (runtime) polymorphism by enabling late binding
- C) It makes a function static
- D) It makes a function inline

Correct Answer: B

Q3. What happens if a base class function is declared `virtual` but is not overridden in the derived class?

- A) Compilation error occurs
- B) The base class version is called when using a base class reference or pointer
- C) Compiler creates a default overriding function in derived class
- D) The derived class cannot be instantiated

Correct Answer: B

Q4. What is an abstract class in C++?

- A) A class that cannot be used
- B) A class with at least one pure virtual function and can only be used as a base class
- C) A class that only contains constructors and destructors
- D) A class that does not have any member functions

Correct Answer: B

Q5. What is true about constructors in inheritance?

- A) The base class constructor is always called first when creating an object of a derived class
- B) The derived class constructor is always called before the base class constructor
- C) The base class constructor is never called when creating a derived class object
- D) None of the others

Correct Answer: A

Q6. What is the output of the following code?

```
class Base{
public:
void hello(){cout << "hi from base";}
};
class Child : Public Base{
public:
void hello(){cout << "hi from child";}
};
int main(){
Child c;
c.hello();
}
```

- A) hi from base
- B) hi from child
- C) hi from basehi from child
- D) hi from childhi from base

Correct Answer: B

Q7 What is the correct way to call a base class parametric constructor from a derived class?

- A) By explicitly calling the base class constructor in the derived class constructor's initialization list using `Base(arguments)`
- B) No need to call it is called automatically
- C) By defining a function in the derived class with the same name as the base class
- D) By using the `virtual` keyword before the base class constructor

Correct Answer: A

Q8. What does the following code output?

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void show() { cout << "Base class" << endl; }
};

class Derived : public Base {
public:
    void show() override { cout << "Derived class" << endl; }
};

int main() {
    Base* bptr;
    Derived d;
    bptr = &d;
    bptr->show();
    return 0;
}
```

- A) Base class
- B) Derived class
- C) Compilation error
- D) Runtime error

Correct Answer: B

Q9. If a derived class inherits from a base class with `protected` access specifier, how are the public members of the base class treated in the derived class?

A) They remain private

B) They become protected and can only be accessed within the derived class and its subclasses

C) They become public and can be accessed anywhere

D) They are inaccessible in the derived class

Correct Answer: B

Q10. What does the following code output?

```
#include <iostream>

using namespace std;

class Base {
public:
    virtual void show() { cout << "Base class" << endl; }
};

class Derived : public Base {
public:
    void show() override { cout << "Derived class" << endl; }
};

int main() {
    Base bptr;
    Derived d;
    bptr = d;
    bptr.show();
    return 0;
}
```

A) Base class

B) Derived class

C) Compilation error

D) Runtime error

Correct Answer: A

Q11. What is the key difference between compile-time and runtime polymorphism?

- a) Compile-time polymorphism uses virtual functions, while runtime polymorphism uses function overloading
- b) Compile-time polymorphism is resolved during program execution, while runtime polymorphism is resolved during compilation
- c) Compile-time polymorphism requires inheritance, while runtime polymorphism does not
- d) Compile-time polymorphism may not require inheritance, while runtime polymorphism requires inheritance

Correct: d)

Q12. What will be the output of the following code?

```
class A {  
    public:  
        void display() {  
            cout << "Class A" << endl;  
        }  
};  
class B : virtual public A {};  
class C : virtual public A {};  
class D : public B, public C {};  
  
int main() {  
    D obj;  
    obj.display();  
    return 0;  
}
```

- a) Class A
- b) Compilation Error
- c) Runtime Error
- d) Garbage Value

Correct: a)

Q13. Which code snippet correctly defines an abstract class?

a)

```
class Animal {  
    public:  
        virtual void sound() { cout << "Noise"; }  
};
```

b)

```
class Animal {  
    public:  
        virtual void sound() = 0;  
};
```

c)

```
class Animal {  
    public:  
        void sound() override;  
};
```

d)

```
class Animal {  
    public:  
        static void sound();  
};
```

Correct: b)

Q14. Complete the blank from the given table:

" _____ in derived class"

Visibility Of Base Class Members	TYPE OF INHERITANCE		
	PUBLIC INHERITANCE	PRIVATE INHERITANCE	PROTECTED INHERITANCE
PUBLIC	PUBLIC in derived class	PRIVATE in derived class	PROTECTED in derived class
PRIVATE	HIDDEN in derived class	HIDDEN in derived class	HIDDEN in derived class
PROTECTED	?	PRIVATE in derived class	PROTECTED in derived class

- a) Public
- b) Hidden
- c) Protected
- d) Private

Correct: c)

Q15. Which keyword is used to EXPLICITLY indicate that a function is overriding a base class virtual function?

- a) virtual
- b) override
- c) abstract
- d) new

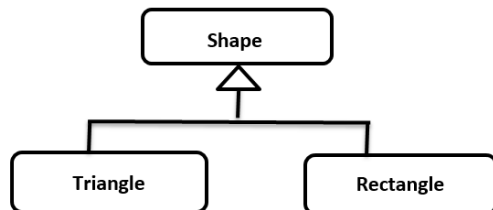
Correct: b)

Q16. How does function overriding differ from function overloading?

- a) Overriding uses the same function name but different parameters, overloading uses the same name and parameters
- b) Overriding occurs in the same class, overloading occurs across classes
- c) Overloading requires the virtual keyword, overriding does not
- d) Overriding is resolved at runtime, overloading at compile-time

Correct: d)

Q17. What type of inheritance is implemented in the figure below?



- a) Multilevel Inheritance
- b) Multiple Inheritance
- c) Hierarchical Inheritance
- d) Hybrid Inheritance

Correct: c)

Q18. What will be the output of the following code?

```
class A{
public:
    void display(){ cout<<"Class A"; }
};
class B: public A {
public:
    void display() override { cout<<"Class B"; }
};
int main(){
    B obj;
    obj.display();
}
```

- a) Class A
- b) Class B
- c) Compilation Error
- d) None of the others

Correct: c)

Q19. Which of the following is a valid use of the final keyword in C++?

- a) To prevent a derived class from overriding a virtual function in the base class
- b) To enforce that a class must be inherited to be instantiated (like an abstract class)
- c) To define an interface class with only pure virtual functions
- d) To enable runtime polymorphism by allowing dynamic binding of member functions

Correct: a)

Q20. What will be the output of the following code?

```
class A{
public:
    A() { cout<<"A constructed! "; }
    ~A(){ cout<<"A destructed! "; }
};

class B: public A{
public:
    B(){ cout<<"B constructed! "; }
    ~B(){ cout<<"B destructed! "; }
};

int main(){
    A* obj = new B();
    delete obj;
}
```

- a) A constructed! B constructed! B destructed! A destructed!
- b) A constructed! A destructed!
- c) A constructed! B constructed! A destructed!
- d) None of the others

Correct: c)